QuantRule — Architecture Description

QuantRule

2026-11-11

Contents

1 QuantRule — Architecture Description **1**
1.1 1. Context . . . 1
1.2 2. Containers . . . 1
1.3 3. Runtime view . . . 2
1.4 4. Component view . . . 2
1.5 5. Data flow . . . 3
1.6 6. Deployment . . . 3
1.7 7. Quality attributes . . . 4
1.8 8. Security and privacy . . . 4
1.9 9. Decisions . . . 4
1.10 10. Evidence and limitations . . . 5
1.11 11. Future work . . . 5

1 QuantRule — Architecture Description

- Status: Final; Phase 4
- Date: 2026-11-11 (UTC)
- Claim type: ADR (Architecture Decision)
- Parent plan: `docs/execution-plans/2026-11-11/588-tauri-platform.md`
- Verification: `docs/reports/2026-11-11/implementation-verification.md`

1.1 1. Context

QuantRule is a local-first research workstation for a solo systematic researcher. The user owns the candles and the machine; the product owns the deterministic pipeline and the local research memory. A static download page distributes Linux installers. A team cloud is future work and is not part of this increment.

The architecture is shaped by four drivers: reproducibility, temporal integrity, auditability, and testability. The style is pipes-and-filters inside a modular monolith, with one local deployable application and local files plus SQLite as its only state.

1.2 2. Containers

Container	Responsibility	Code
Desktop UI	Workspace, data, configs, model run, memory, export/backup, settings	`apps/desktop`
Application service	Use-case orchestration, file access, workspace, datasets, config, model run, export, memory	`crates/quantrule-app`
Data engine	Candle normalization, timeframe ladder, ordering, gaps	`crates/quantrule-data`
Model engine	Readings, states, levels, events, progression, context	`crates/quantrule-model`
Backtest engine	Strategy, execution, portfolio, benchmark	`crates/quantrule-backtest`
CLI shell	Headless command surface and explicit source fetch	`crates/quantrule-shell`
Local research memory	SQLite workspaces, datasets, configs, runs, metrics, claims, notes	`crates/quantrule-app/src/memory/`
Download page	Static Linux installer distribution and checksums	`site/`

The desktop UI and the CLI are adapters over the same application service. The engines are pure and depend one way. The UI does not compute an indicator, a rule, a fill, or an accounting value.

1.3 3. Runtime view

```
Desktop UI (Tauri + Svelte)
  +- Tauri commands
       +- quantrule-app
            +- workspace / dataset / config
            +- model run
            +- export
            +- SQLite research memory

CLI shell (quantrule)
  +- quantrule-app config and pipeline
       +- backtest, strategy, execution, portfolio, analytics

Static download page
  +- AppImage / deb + SHA-256 checksums
```

The desktop app covers workspace, data, configs, model run, memory, export/backup, and settings. Backtest, strategy, execution, portfolio, qualification, and analytics remain CLI operations until the service owns them.

1.4 4. Component view

Component	Responsibility	Key code
Workspace	Resolve a local root; list sorted configs and datasets	`crates/quantrule-app/src/workspace.rs`
Dataset	Open one M1 CSV and report its extent	`crates/quantrule-app/src/dataset.rs`
Config	Strictly load and cross-check run, data, model, execution, portfolio	`crates/quantrule-app/src/config/`
Pipeline	Load the ladder and compute the model	`crates/quantrule-app/src/pipeline.rs`
Service	Shared use cases and validated run summary	`crates/quantrule-app/src/service.rs`
Export	Versioned, deterministic run-summary JSON	`crates/quantrule-app/src/export.rs`
Memory	Schema, CRUD, export/import, backup/restore	`crates/quantrule-app/src/memory/`
Tauri commands	One command per service operation	`apps/desktop/src-tauri/src/commands.rs`
Svelte shell	Panels, state, typed IPC, fixture backend	`apps/desktop/src/`
Distribution	Bundle, stage, checksum, static page	`site/`

The application service owns file access and path resolution. The engines receive validated Rust values and never parse a file. The shell owns command parsing, terminal output, the clock, and explicit source fetch.

1.5 5. Data flow

```
local M1 candles
  -> data engine (normalization, ladder)
  -> model engine (readings, states, levels, events, progression, context)
  -> strategy (human rule)
  -> execution (fills)
  -> portfolio (accounting)
  -> evidence + local research memory
  -> desktop UI / CLI
```

The model publishes per-timeframe results only. Cross-timeframe reporting views are computed by the reporting consumer. The strategy composes its own as-of context from the published events and states; it never computes an indicator or reads cash/P&L.

The research memory records the validated run, its workspace, dataset, and config, plus typed metrics, claims, and notes. It is outside the signal path.

1.6 6. Deployment

Surface	Artifact	Evidence
Desktop	`QuantRule_0.9.0_amd64.AppImage`	Xvfb launch, offline launch
Desktop	`QuantRule_0.9.0_amd64.deb`	Ubuntu 24.04 container install + launch
CLI	`quantrule` binary	workspace tests and command tests
Distribution	Static `site/` page and `checksums.txt`	`site/check.py`
State	Local files and SQLite	memory tests

There is no server, database service, account, or cloud runtime. The download page is distribution only.

1.7 7. Quality attributes

Attribute	Design response	Evidence
Reproducibility	Pure engines, fixed accumulation order, ordered collections, provenance	Repeated model bundles and `run check` byte-identical
Temporal integrity	Availability bars, causality tests, as-of joins	Model, event, and strategy boundary tests
Auditability	Artifact bundles, config hash, data version, research memory	Provenance and memory tests; verification report
Testability	Unit, golden, causality, determinism, shell, UI, E2E	1,476 Rust tests; 7 frontend; 1 Playwright E2E; 4 desktop backend
Modularity	One crate per engine plus application service; one-way edges	Architecture guard and structural checker
Deployability	Tauri bundles and static page	Clean deb install; AppImage launch
Evolvability	Versioned run, config, and memory schemas	Migration tests; versioned contracts
Local security	No account; no engine network; explicit fetch only	Offline AppImage launch; source scan
Cost	Local compute; static page	No cloud runtime

1.8 8. Security and privacy

- No account, login, subscription, or cloud data store.
- No engine network call. The only network path is the explicit CLI `fetch`.
- Candles, configs, strategies, and research memory stay local.
- The Tauri capability file grants only `core:default`; the app has no shell or file-system plugin permission.
- The download page publishes installers and checksums only.

1.9 9. Decisions

Decision	Record
Local-first Tauri platform; team cloud deferred	`docs/adr/2026-11-11/adopt-local-first-tauri-platform`
Four-stage board, claim types, review gate	`docs/execution-plans/2026-11-11/590-workflow-v2.md`
Run v3 and portfolio constraints	`docs/config/2026-11-11/run-v3-and-portfolio-constrai`
Research memory v1	`docs/config/2026-11-11/research-memory-v1.md`
Architecture baseline	`docs/architecture/README.md`

1.10 10. Evidence and limitations

The Phase 3 gate is recorded in `docs/reports/2026-11-11/implementation-verification.md`: full Rust and desktop gates, two bundles, clean Debian install, offline AppImage launch, byte-identical model runs, and research-memory round-trips.

Limitations:

- The browser E2E drives the Vite build with a fixture backend; the native smoke proves the Tauri shell launches but does not drive the UI.
- The clean install runs in a container, not bare-metal Ubuntu.
- The UI does not run backtests or analytics; those remain CLI operations.
- No tag or GitHub release exists in this phase.

1.11 11. Future work

A collaborative team cloud is a future increment. It would add a control plane for accounts, config/run metadata, roles, and verification, but not candles or cloud backtesting. It requires a new ADR and characteristics review.